Three tools and one reliable sender

An abridged version of the three-tools report

A message sent over an unreliable network can be lost. The usual remedy is for the sender to keep a copy of each message and resend it until the receiver acknowledges it. That remedy is correct, but it can also hurt the system it protects: when acknowledgements are late, the sender cannot tell a lost message from a slow one, so it resends messages that have already arrived, and the copies compete with new work for the receiver's capacity.

This report follows such a sender through three tools, each of which asks a different question about it. **Hydrofrac** asks whether the protocol is correct when messages are lost and machines crash. **Cress** asks whether delay makes the protocol do work that its input did not require. **Sedge** asks whether the deployed protocol recovers from a burst of load when the receiver's CPU is capped from outside. The naive protocol passes the first test and fails the other two. We then add a token bucket that limits resends. Only hydrofrac notices that one plausible bucket can lose data forever, and a bucket that refills from both a timer and acknowledgements passes all three tools.

The programs are written in Hydro, a Rust framework in which a distributed program is a *dataflow*: a graph of operators (filters, joins, unions) through which streams of values flow, split across machines that are connected by network channels.

1. Correctness: from `send` to `reliable_send`

Plain `send` over a lossy link

Hydrofrac applies lineage-driven fault injection (LDFI; Alvaro, Rosen and Hellerstein, SIGMOD 2015) to Hydro programs, running them in Hydro's deterministic simulator. The simplest example shows how it works. A sender passes value 1 to Hydro's ordinary `send` over a channel that may lose messages, and a receiver delivers whatever arrives. LDFI starts from a *successful* run, with no faults, in which the good outcome occurs: the receiver delivers value 1. It then asks whether some combination of failures could falsify that outcome. To answer, it reads the outcome's *lineage*, which is the record of why the outcome happened. Here the lineage is short: the receiver holds 1 because one message carrying 1 crossed the `data` channel. A fault matters only if it cuts that lineage, so LDFI computes the smallest sets of crashes and lost messages that would do so. The smallest is the loss of that one message. Hydrofrac injects the loss and reruns the program, and the receiver never holds 1. That run is a counterexample, found after two fault sets, and Figure 1 shows it as a Lamport diagram, with one timeline per machine and one arrow per message.

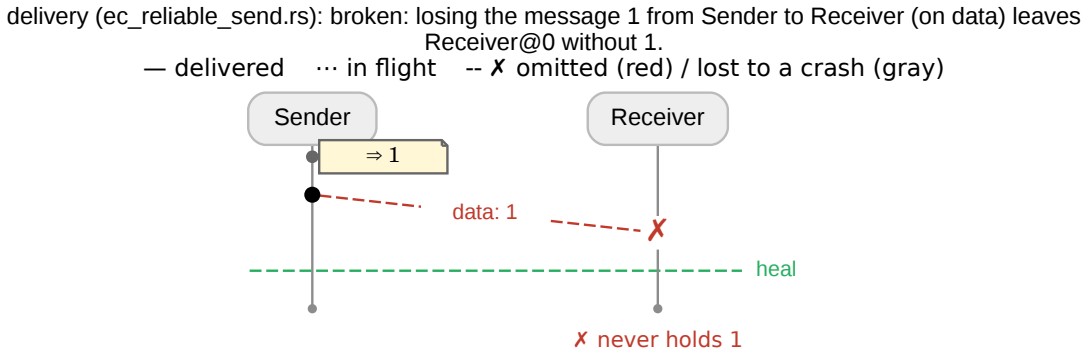


Figure 1: Hydrofrac's counterexample for plain `send` over a lossy link. Each horizontal line is a machine and each arrow is a message, and a red arrow ending in ✗ is a lost message. The sender's only copy of value 1 is lost on the `data` channel before the network heals, and the receiver never holds 1.

The property checked here is validity: *every value handed to a sender that never crashes is delivered at every receiver that never crashes*. If either machine crashes, the property makes no demand, since a crashed sender may never have sent the value and a crashed receiver cannot deliver it. Hydrofrac states a property as an implication from one collection to another:

```
vals.implies(          // vals: the values handed to the sender, the "if" side of the rule
    "delivery",        // the name that appears in reports
    &delivered,        // the receiver's delivered values, the "then" side of the rule
    q!(|v| v),         // the key of an input value: the value itself
    q!(|v| v),         // the key of a delivered value: the value itself
    ASide::Correct,    // count only the inputs of senders that never crash
    BSide::AllCorrect, // and require every receiver that never crashes to deliver each of them
);
```

Resending until acknowledged

The fix is `reliable_send`, a drop-in replacement for `send`. The sender puts each new value on the wire at once and adds it to an *outstanding set*. The receiver answers every arrival, including duplicates, with an *acknowledgement* (an *ack*), a short message saying that it has the value, and each ack removes its value from the outstanding set. A timer fires once per second, and each time it fires the sender resends every value that is still outstanding. The receiver delivers each value once and discards duplicates.

Hydrofrac asks the same question of `reliable_send` and gets a different answer. When it cuts the first copy of a value, the rerun still delivers the value, because a later tick resent it. The outcome had a redundant support, which joins the lineage for the next round of the search. Faults are allowed only before the network heals, and afterwards the timer keeps firing, so every fault set leaves some copy that gets through, and when no fault set within its bounds can cut every support, hydrofrac issues a certificate. With one value and a resend that may itself be lost, it certified delivery after 112 fault sets in about a second, and with three values, any of whose first copies and acks may be lost in any interleaving, after 576 fault sets in 74 seconds. Plain `send` is the control for this experiment: the same harness breaks it with a single loss, so the certificate for `reliable_send` is not an artifact of a search that cannot find losses.

2. Two more questions about the same protocol

Cress: does delay turn into extra work? Cress also runs the program in the simulator, but it never loses a message. Instead it holds back one input batch, such as the incoming acks, for longer and longer, from one round of the simulation up to 999 rounds, and it compares the number of messages sent with that of a run in which nothing is held. If the extra work stops at a ceiling, delay is harmless, and if it keeps growing with the hold, the program turns delay into load. For `reliable_send`, holding the acks makes the extra work grow without a ceiling. At a hold of 999 rounds the sender sends 232,252 more messages than in the unheld run, which is 248 extra messages per round of delay and a multiplier of 930. The rate of 248 is the size of the outstanding set, since every value sent after the hold began is resent on every tick.

Sedge: does the deployed program recover from a burst? Sedge deploys the program in Docker containers and imposes a capacity from outside with a CPU quota, so the program contains no capacity code. It first calibrates μ, the receiver’s capacity, which is the highest steady rate of new values per second that the receiver keeps up with. It then offers a steady load λ = μ/f for several factors f, so a larger f means a lighter load. The load is open loop, meaning that values arrive on a schedule whether or not earlier ones have been delivered. Each run has a 10-second baseline, a 5-second burst at twice μ, and a 45-second tail, and Sedge compares the tail with the baseline to report that the program RECOVERED, is still DRAINING, or COLLAPSED. Its deployments run over plain TCP and lose nothing. The configuration is an attribute on the function under test:

```
#[cfg_attr(feature = "sedge", sedge::overload_check(
    name = two_per_round,
    T = super::rpc_retry::Payload<4096>,   // 4 KiB values
    load = values,                         // the input that receives the open-loop load
    mu = 500,
    cpus(Sender = none, Receiver = 0.01),  // the receiver gets 1% of a core
    cpu_period_us = 100_000,
    timers(resend_ticks = std::time::Duration::from_secs(1)),
))]
```

With one percent of a core, the receiver’s capacity is μ = 500 values per second. We ran two trials at each of f = 2, 4, 8, 16, 32 and 64, so the steady load ranged from 250 down to 8 values per second, and eleven of the twelve trials collapsed. Figure 2

shows a typical one. After the burst the sender writes about 800 messages per second at every load, while the receiver, throttled in 85–100% of its CPU periods, delivers only 2–11% of that traffic as new values. The rest is resends.

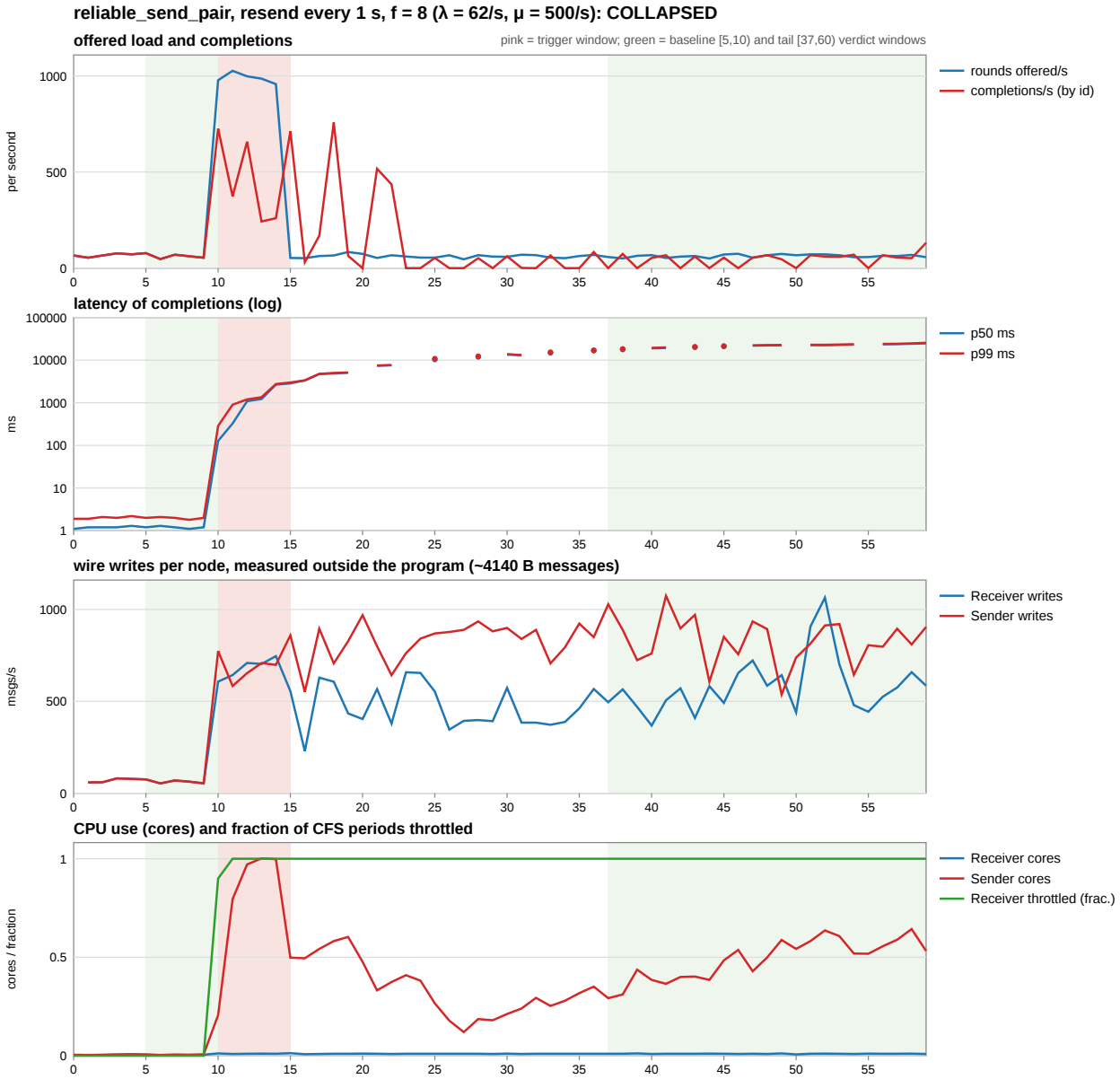


Figure 2: Sedge's trace of `reliable_send` (V0) at f = 8, which offers λ = 62 values per second, with a one-second resend timer. The pink band is the burst, and the green bands are the baseline and tail windows that the verdict compares. After the burst, completions fall to almost nothing, latency climbs past 10 s, the sender keeps writing about 800 messages per second, and the receiver stays throttled.

Figure 3 is the control for Figure 2. It runs the same program with the same receiver quota, the same one-second resend timer and the same steady load of λ = 62 values per second, but the "burst" is set to the baseline rate, so the load stays flat. Sedge reports RECOVERED. Every value offered in the baseline and tail windows completed, with a median latency of 1.2 ms in both, and no value was outstanding at the end. The receiver used 0.003 cores and was throttled in 3% of its CPU periods, and the sender wrote about as many messages per second as the load offered for the whole run. The steady load alone therefore leaves the protocol healthy, and the collapse in Figure 2 comes from the burst and the resend storm that it starts. This control is a single trial.



Figure 3: The control for Figure 2: `reliable_send` at f = 8 with a one-second resend timer and a flat load of 62 values per second. The pink band marks where the burst falls in the other runs, but here the load does not rise. Completions track the offered load, median latency stays near 1 ms, and the receiver is barely throttled. The vertical axes are scaled to this run, so they differ from those of Figure 2.

Why it collapses. The burst, and not the steady load, sets the size of the storm. During the burst, values arrive at twice the rate that the receiver can acknowledge, so the outstanding set grows to about 2,000 values. Every tick then resends all of them, so the sender asks for at least 2,000 messages per second from a receiver that can absorb about 800. While the receiver works through resends, acks stay late and the outstanding set stays large, so the next tick resends it all again. Stretching the resend period to 20 seconds cuts this demand by the same factor, and in one trial each at f = 2 and f = 8 the protocol then recovered within about three seconds of the burst. A slow timer is a blunt fix, however, since it also slows recovery from every real loss twentyfold.

3. A budget for resends

A *token bucket* is the standard way to limit a rate. The bucket holds up to B tokens, every resend spends one, a resend that finds the bucket empty waits, and some event refills the bucket. First copies of new values stay free. Our buckets start full with B = 10 tokens, and the variants differ only in where tokens come from. V2 earns one token per four acks that clear an outstanding

value, and V3, the *hybrid*, also earns one token per timer tick. The bucket lives in the same dataflow as the sender, and resends go round robin, so no value waits forever while tokens keep coming. Its core is a few lines:

```
let available = tokens.zip(tick_grant).zip(ack_grant)
    .map(q!(move |((saved, by_tick), by_ack)| (saved + by_tick + by_ack).min(capacity)));
// ... resend the first `available` outstanding values, and move them to the back of the queue ...
tokens = available.zip(spent).map(q!(|(available, spent)| available - spent));
```

Table 1: The verdicts of the three tools, with the unbudgeted `reliable_send` labelled V0. The budgeted variants ran in Sedge at f between 1.1 and 16, with μ pinned at 500 so that every variant saw the same load.

variant	hydrofrac: delivery under loss and crashes	Cress: extra messages at a 999-round ack hold	Sedge: recovery from the burst
V0, no bucket	Holds.	232,252, still growing at 248 per round.	11 of 12 trials collapsed.
V2, refilled by acks	**Broken: a value can be lost forever.**	10, a fixed ceiling.	12 of 12 trials recovered.
V3, refilled by acks and the timer	Holds.	1,007, growing at one per round.	12 of 12 trials recovered.

Table 1 collects the verdicts. A third bucket (V1), which the timer alone refills, also passes all three tools in this setting and differs from the hybrid only on a lossy link, as described below.

The bucket that starves. By the measures of Cress and Sedge, the ack-refilled bucket V2 looks like the best design, since its extra work under delay stops at a ceiling of ten messages and it recovers from every burst. Hydrofrac finds that it can lose a value forever. Figure 4 shows the counterexample for a test bucket of one token, found after three fault sets and 15 ms. The first copy of value 1 is lost, the first tick spends the only token on a resend, and that copy is lost too. No ack can now arrive, so no token is ever earned, and the value stays outstanding forever. Two lost messages suffice, and no crash is needed. For every bucket from one to four tokens, hydrofrac found the same starvation with exactly B + 1 lost messages, the first copy and the B resends that the bucket pays for. For our ten-token bucket the pattern predicts 11 lost messages, a search beyond hydrofrac's current reach. Cress and Sedge cannot see the problem, because neither of them loses messages, and without loss the acks keep refilling the bucket.

The bucket that works. The hybrid bucket V3 passes all three tools. Hydrofrac certifies it with the same harnesses and the same number of fault sets as V0, and a hybrid bucket of one token delivers a single value under the losses that starve V2, because the timer earns the token for the next resend. Cress measures one extra message per round of delay, which is the timer's refill rate, whatever the load. Sedge finds that it recovers in all twelve trials. Figure 5 shows V3 on exactly the same arrivals as the collapsed run of Figure 2. Its median latency settles back near its baseline within three seconds of the burst, the sender writes 63 messages per second in the tail, which is the offered load, and the receiver is throttled in 4% of its CPU periods.

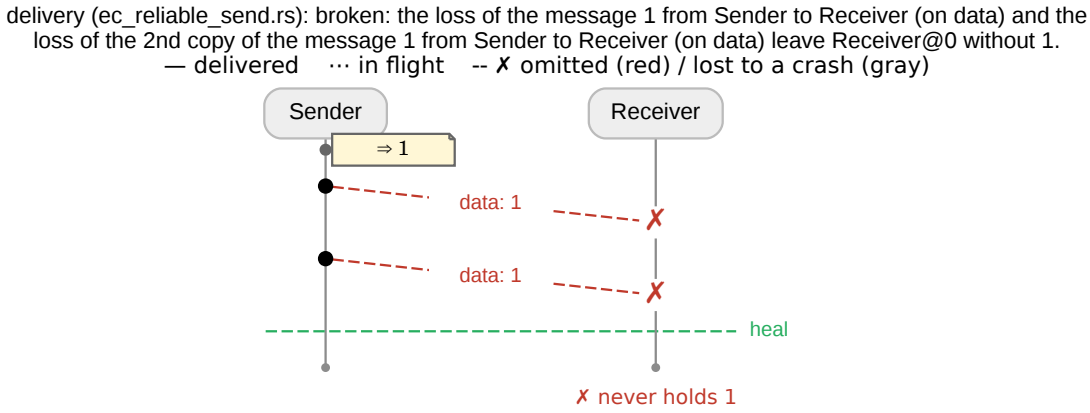


Figure 4: Hydrofrac's counterexample for an ack-refilled bucket of one token (V2). The first copy of value 1 and the one resend that the token pays for are both lost on the `data` channel. No ack ever returns to earn another token, and after the network heals the receiver never holds value 1.

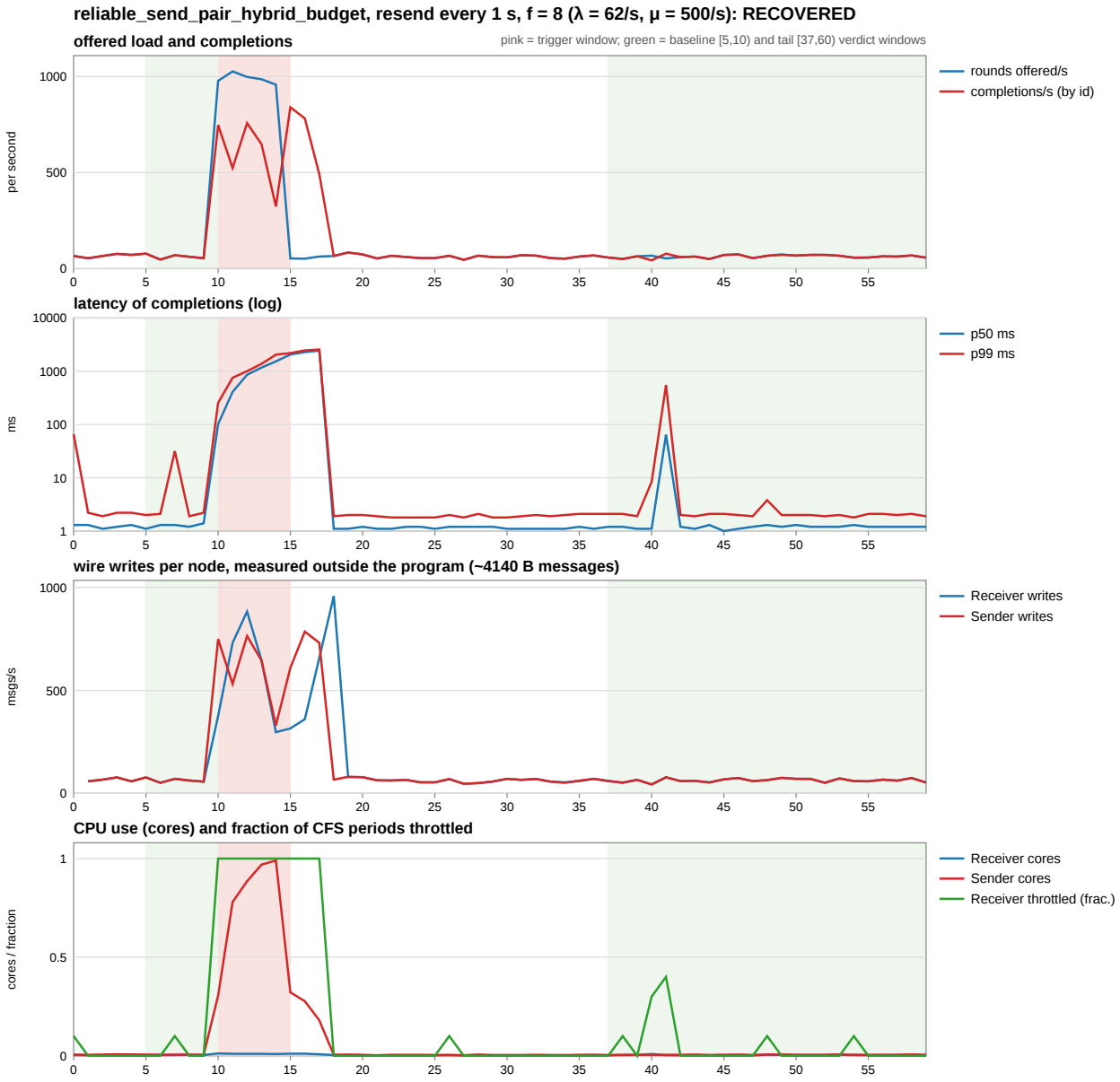


Figure 5: Sedge's trace of the hybrid bucket (V3) at f = 8 with a one-second resend timer, on the same seeded arrivals and the same receiver quota as Figure 2. The latency spike during the burst clears within about three seconds, and in the tail the sender writes only the offered load.

A twist on a lossy link. In a final experiment we made Sedge's transport lossy, by putting a queue of 64 messages in front of every connection and dropping any send that finds its queue full. Under this tail-drop loss the ranking changes. V0 recovered best, delivering all of its lost values within 28 seconds of the burst, because the full queues shed its resend storm inside the sender. The hybrid bucket recovered lost values at about nine per second. The bucket refilled by the timer alone resent only about one value per second and, in a four-minute run, never caught up with the values that it lost. The right budget therefore depends on the transport: the collapse in Figure 2 needs send buffers that queue every message, and over a lossy transport a stingy bucket slows recovery.

4. What the tools found together

Each tool caught something that the other two missed. Hydrofrac alone showed that the ack-refilled bucket can lose a value forever, Cress alone measured how each design converts delay into work, and Sedge alone showed that the unbudgeted protocol collapses under a capped receiver while the budgeted variants recover. Of the designs we tried, only the hybrid bucket did well on every test. The separation between V0 and the budgeted variants is unlikely to be noise: the variants ran on

identical seeded arrivals, and in all eight pairs of trials with the same factor and seed, V0 did worse than each budgeted variant, which gives p = 0.0078 by a sign test.

5. Limits

- Every Sedge result comes from one 16-core host that also ran a background process, with two trials per setting and two fixed seeds, and V0 ran in an earlier session than the budgeted variants. The results show a clear separation on this host and do not establish general collapse thresholds.
- Hydrofrac's certificates describe the simulator's model of the program and its test harness, within bounded searches. For rate-limited variants the verdict also depends on how long the harness lets the protocol recover.
- Starvation of the ten-token ack-refilled bucket is predicted by the pattern that hydrofrac demonstrated for one to four tokens, and no run has demonstrated it directly.
- Cress never loses messages, and Sedge loses them only in the lossy-transport experiment, which used one loss model and few seeds.
- The bucket parameters were taken from an existing corpus and were not tuned.

The full report, `docs/design/reliable-send-three-tools.md`, names every test and run, and it gives the trace analysis behind the diagnosis of the collapse, the sweeps over bucket sizes and input rates, the complete lossy-transport experiment, and instructions to reproduce each result.